

ISLAMIC UNIVERSITY OF TECHNOLOGY



MACHINE LEARNING LAB

CSE 4622

Lab 2: Data Loading, Preprocessing, and Exploratory Data Analysis

Author:

Ishmam Tashdeed
CSE, IUT

Contents

1	Introduction	3
2	Loading Data	5
2.1	Reading a CSV File	5
2.2	Previewing the Data	6
2.3	Inspecting Data Types and Structure	6
3	Descriptive Statistics	7
3.1	Summary Statistics for Numerical Columns	7
3.2	Summary Statistics for Categorical Columns	8
4	Handling Missing Values	9
4.1	Detecting Missing Values	9
4.2	Strategies for Handling Missing Values	10
4.3	Dropping Columns with Excessive Missing Values	11
4.4	Imputing Missing Values	11
4.4.1	Numerical Column: Age (fill with median)	11
4.4.2	Categorical Column: Embarked (fill with mode)	12
5	Encoding Categorical Features	12
5.1	Label Encoding	12
5.2	One-Hot Encoding	13
6	Feature Scaling	15
6.1	Min-Max Normalization	15
6.2	Standardization (Z-score Scaling)	16
6.3	Comparing the Two Methods Visually	17
7	Exploratory Data Analysis	18
7.1	Distribution of Individual Features	18
7.1.1	Histograms	18
7.1.2	KDE Plots (Kernel Density Estimation)	19
7.2	Categorical Feature Distributions	19
7.2.1	Bar Charts	19
7.3	Box Plots and Outlier Detection	21
7.4	Correlation Analysis	22
7.4.1	Correlation Heatmap	22

7.4.2	Scatter Plots	23
7.5	Pair Plot	24
7.6	Violin Plots	24
8	Additional Resources	25
9	Tasks	26
10	Submission Instructions	27

1 Introduction

Before we can train any machine learning model, we must deeply understand the data we are working with. Feeding raw, unprocessed data directly into a model almost always leads to poor results. This lab introduces the full pipeline that comes *before* modelling: loading data, cleaning it, transforming it into a usable form, and exploring it visually and statistically.

This pipeline is collectively known as **Exploratory Data Analysis (EDA)** and **Data Preprocessing**. By the end of this lab, you will be able to:

- Load a CSV dataset into Python using `pandas`.
- Inspect and understand the structure of a dataset.
- Detect and handle missing values.
- Encode categorical (non-numeric) features.
- Scale numerical features.
- Compute descriptive statistics.
- Create informative visualizations to reveal patterns, distributions, and relationships in data.

Dataset for This Lab

Throughout this lab, we will use the **Titanic dataset**, a classic and beginner-friendly dataset that contains information about passengers aboard the RMS Titanic. It includes a mix of numerical and categorical features, and has missing values.

Download the dataset from:

<https://www.kaggle.com/datasets/yasserh/titanic-dataset>

Save it as `titanic.csv` in the same folder as your Colab notebook, or upload it directly to your Colab session using the file panel on the left. Alternatively, you can load it directly from the URL in your code (shown in Section 2).

Table 1 describes the columns in the dataset.

Table 1: Description of Titanic dataset columns.

Column	Type	Description
PassengerId	Integer	Unique identifier for each passenger
Survived	Integer	Survival label: 0 = No, 1 = Yes (target variable)
Pclass	Integer	Ticket class: 1 = 1st, 2 = 2nd, 3 = 3rd
Name	String	Full name of the passenger
Sex	String	Gender of the passenger
Age	Float	Age in years
SibSp	Integer	Number of siblings/spouses aboard
Parch	Integer	Number of parents/children aboard
Ticket	String	Ticket number
Fare	Float	Passenger fare paid
Cabin	String	Cabin number
Embarked	String	Port of embarkation: C, Q, or S

Setting Up Your Environment

Open a new notebook in Google Colab (<https://colab.research.google.com>) and name it ML_Lab2_YOUR_STUDENT_ID. Run the following cell first to install and import everything you need:

Code 1: Importing all required libraries.

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 from sklearn.preprocessing import LabelEncoder,
   OneHotEncoder
7 from sklearn.preprocessing import MinMaxScaler,
   StandardScaler
8
9 # Make plots appear inline in the notebook
10 %matplotlib inline
11
12 # Optional: use a cleaner plot style
13 sns.set_theme(style="whitegrid")

```

```
14  
15 print("All libraries loaded successfully!")
```

Note

If you see a `ModuleNotFoundError`, install the missing library by running `!pip install library_name` in a new cell. All libraries used here come pre-installed in Google Colab.

2 Loading Data

The first step in any data science project is loading the data into memory. We use the **pandas** library for this. **pandas** stores tabular data in a structure called a **DataFrame**. Think of it as a powerful, programmable spreadsheet.

2.1 Reading a CSV File

A *CSV (Comma-Separated Values)* file stores tabular data as plain text where each row is a data record and each column is separated by a comma. We load it with `pd.read_csv()`.

Code 2: Loading the dataset from a CSV file.

```
1 # Load from a CSV file  
2 file_path = "titanic.csv"  
3 df = pd.read_csv(file_path)  
4  
5 print("Dataset loaded successfully!")  
6 print(f"Shape: {df.shape}") # (rows, columns)
```

Expected Output

```
Dataset loaded successfully!  
Shape: (891, 12)
```

The `shape` attribute tells us the dataset has **891 rows** (passengers) and **12 columns** (features).

2.2 Previewing the Data

Always look at the first few rows of a dataset before doing anything else. This gives you a quick sense of what the data looks like.

Code 3: Previewing the first and last rows.

```
1 # Display the first 5 rows
2 df.head()
```

Code 4: Viewing the last 5 rows.

```
1 # Display the last 5 rows
2 df.tail()
```

Code 5: Viewing a random sample of rows.

```
1 # View a random sample of 5 rows (useful for large
  datasets)
2 df.sample(5, random_state=42)
```

2.3 Inspecting Data Types and Structure

Code 6: Inspecting column names, types, and memory usage.

```
1 # Column names and their data types
2 df.info()
```

Expected Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
# Column Non-Null Count Dtype
-----
0 PassengerId 891 non-null int64
1 Survived 891 non-null int64
2 Pclass 891 non-null int64
3 Name 891 non-null object
4 Sex 891 non-null object
5 Age 714 non-null float64
```

```
6 SibSp 891 non-null int64
7 Parch 891 non-null int64
8 Ticket 891 non-null object
9 Fare 891 non-null float64
10 Cabin 204 non-null object
11 Embarked 889 non-null object
```

Note

The Non-Null Count column immediately reveals missing data. Notice that **Age** has only 714 non-null values (meaning 177 are missing), **Cabin** has only 204 (687 missing!), and **Embarked** is missing 2 values. We will address these in Section 4.

Code 7: Listing all column names.

```
1 # Get a list of all column names
2 print(df.columns.tolist())
```

Code 8: Checking data types of each column.

```
1 # Data types of every column
2 print(df.dtypes)
```

3 Descriptive Statistics

Before cleaning, it helps to understand the basic statistical properties of your data. This step reveals the range, central tendency, and spread of numerical features at a glance.

3.1 Summary Statistics for Numerical Columns

Code 9: Computing summary statistics.

```
1 # Summary statistics for all numerical columns
2 df.describe()
```

Expected Output

	PassengerId	Survived	Pclass	Age	Fare
count	891.00	891.00	891.00	714.00	891.00
mean	446.00	0.38	2.31	29.70	32.20
std	257.35	0.49	0.84	14.53	49.69
min	1.00	0.00	1.00	0.42	0.00
25%	223.50	0.00	2.00	20.12	7.91
50%	446.00	0.00	3.00	28.00	14.45
75%	668.50	1.00	3.00	38.00	31.00
max	891.00	1.00	3.00	80.00	512.33

The table above gives us, for each numerical column:

- **count** — the number of non-null values.
- **mean** — the arithmetic average.
- **std** — the standard deviation (how spread out values are).
- **min / max** — the smallest and largest values.
- **25%, 50%, 75%** — the quartiles (25% is Q_1 , 50% is the median, 75% is Q_3).

3.2 Summary Statistics for Categorical Columns

Code 10: Describing non-numeric (object) columns.

```
1 # Describe categorical columns
2 df.describe(include='object')
```

Code 11: Counting unique values in categorical columns.

```
1 # Count unique values per column
2 print(df.nunique())
```

Code 12: Value counts for a categorical column.

```
1 # How many passengers are in each ticket class?
2 print(df['Pclass'].value_counts())
```

```

3
4 # How many survived vs. did not survive?
5 print(df['Survived'].value_counts())

```

Expected Output

```

3 491
1 216
2 184
Name: Pclass, dtype: int64

0 549
1 342
Name: Survived, dtype: int64

```

4 Handling Missing Values

Real-world data is almost always incomplete. Missing values, if left unaddressed, will cause errors in most machine learning algorithms. Your first job is to *detect* them, then make a deliberate decision about how to *handle* them.

4.1 Detecting Missing Values

Code 13: Detecting missing values across all columns.

```

1 # Count the number of missing values per column
2 print(df.isnull().sum())

```

Expected Output

```

PassengerId 0
Survived 0
Pclass 0
Name 0
Sex 0
Age 177
SibSp 0

```

```
Parch 0
Ticket 0
Fare 0
Cabin 687
Embarked 2
dtype: int64
```

Code 14: Visualising missing data as a heatmap.

```
1 plt.figure(figsize=(10, 4))
2 sns.heatmap(df.isnull(), cbar=False, cmap='viridis',
3             yticklabels=False)
4 plt.title("Missing Value Map (yellow = missing)")
5 plt.tight_layout()
6 plt.show()
```

Note

The heatmap gives you a bird's-eye view of where missingness is concentrated. Long yellow streaks indicate columns with heavy missing data (here, Cabin).

4.2 Strategies for Handling Missing Values

There is no single “correct” strategy. The choice depends on *how much* data is missing and *why* it is missing.

Table 2: Common strategies for handling missing values.

Strategy	When to Use
Drop the column	>50% missing; column is not important
Drop the row	Very few rows are missing
Fill with mean	Numerical; roughly symmetric distribution
Fill with median	Numerical; skewed distribution or outliers
Fill with mode	Categorical column
Fill with constant	Domain knowledge suggests a default value

4.3 Dropping Columns with Excessive Missing Values

The `Cabin` column is missing for over 77% of passengers. Rather than trying to impute it, we drop it.

Code 15: Dropping the `Cabin` column.

```
1 df = df.drop(columns=['Cabin'])
2 print(f"Columns remaining: {df.shape[1]}") # Should
   be 11
```

We also drop columns that will not be useful as features: `PassengerId`, `Name`, and `Ticket` are unique identifiers or free text that carry no predictive signal for our purpose.

Code 16: Dropping identifier columns.

```
1 df = df.drop(columns=['PassengerId', 'Name', 'Ticket'])
2 print(df.columns.tolist())
```

Expected Output

```
['Survived', 'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare',
 'Embarked']
```

4.4 Imputing Missing Values

4.4.1 Numerical Column: Age (fill with median)

We use the **median** rather than the mean because the `Age` column may be slightly skewed (older passengers who could afford higher-class tickets).

Code 17: Filling missing `Age` values with the median.

```
1 age_median = df['Age'].median()
2 print(f"Median age: {age_median}")
3
4 df['Age'] = df['Age'].fillna(age_median)
5
6 # Verify no more missing values in Age
7 print(f"Missing in Age after imputation: {df['Age'].
   isnull().sum()}")
```

Expected Output

```
Median age: 28.0  
Missing in Age after imputation: 0
```

4.4.2 Categorical Column: Embarked (fill with mode)

Only 2 rows are missing **Embarked**. We fill them with the most common port.

Code 18: Filling missing **Embarked** values with the mode.

```
1 embarked_mode = df['Embarked'].mode()[0]  
2 print(f"Most common port: {embarked_mode}")  
3  
4 df['Embarked'] = df['Embarked'].fillna(embarked_mode)  
5  
6 print(f"Missing in Embarked after imputation: {df['  
    Embarked'].isnull().sum()}")
```

Expected Output

```
Most common port: S  
Missing in Embarked after imputation: 0
```

5 Encoding Categorical Features

Machine learning models operate on numbers. Columns that contain text (strings) such as **Sex** and **Embarked** must be converted to numerical representations. This process is called **encoding**.

At this point, our dataset has two categorical columns:

- **Sex** — two categories: **male**, **female**.
- **Embarked** — three categories: **S**, **C**, **Q**.

5.1 Label Encoding

Label Encoding replaces each category with an integer. For example: **female** $\rightarrow$ 0, **male** $\rightarrow$ 1.

- **Best for:** Binary columns (exactly 2 categories) or ordinal categories (where order matters, e.g., Low < Medium < High).
- **Caution:** Applying it to a nominal column with 3+ categories implies an ordering that does not exist. For example, encoding {Red, Green, Blue} as {0, 1, 2} implies Blue > Red, which is meaningless.

Code 19: Label encoding the Sex column.

```

1 le = LabelEncoder()
2 df['Sex_encoded'] = le.fit_transform(df['Sex'])
3
4 # Check the mapping
5 print("Classes:", le.classes_)      # ['female', 'male']
6 print(df[['Sex', 'Sex_encoded']].head())

```

Expected Output

Classes: ['female' 'male']

	Sex	Sex_encoded
0	male	1
1	female	0
2	female	0
3	female	0
4	male	1

Remember

Label Encoding is fine for **Sex** because it is binary. Applying it to **Embarked** would be a mistake — we would use One-Hot Encoding for that instead (see Section 5.2).

5.2 One-Hot Encoding

One-Hot Encoding (OHE) creates one new binary column for each category. For **Embarked** (3 categories), it creates 3 new columns: **Embarked_C**, **Embarked_Q**, **Embarked_S**. Each row has exactly one 1 and the rest are 0.

- **Best for:** Nominal columns with no natural ordering, especially with 3 or more categories.
- **Note:** Use `drop_first=True` to avoid the *dummy variable trap* (perfect multicollinearity), which can harm linear models.

Code 20: One-hot encoding the Embarked column using pandas.

```
1 # pd.get_dummies is the easiest way to one-hot encode
2 embarked_dummies = pd.get_dummies(df['Embarked'],
3   prefix='Embarked', drop_first=True)
4 print(embarked_dummies.head())
```

Expected Output

	Embarked_Q	Embarked_S
0	False	True
1	False	False
2	False	True
3	False	True
4	False	True

Code 21: Concatenating the encoded columns and dropping the originals.

```
1 # Attach the new dummy columns to the main dataframe
2 df = pd.concat([df, embarked_dummies], axis=1)
3
4 # Drop the original text columns (no longer needed)
5 df = df.drop(columns=['Sex', 'Embarked'])
6
7 print(df.columns.tolist())
8 print(df.head())
```

Expected Output

```
['Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare',
'Sex_encoded', 'Embarked_Q', 'Embarked_S']
```

Note

When to use which encoding?

- Binary category (2 values): use **Label Encoding**.
- Nominal category (≥ 3 values, no order): use **One-Hot Encoding**.
- Ordinal category (≥ 3 values, order matters): use **Label Encoding** or **Ordinal Encoding**.

6 Feature Scaling

Numerical features in a dataset often have very different units and ranges. For example, in our dataset, **Age** ranges from 0 to 80, while **Fare** ranges from 0 to 512. Many machine learning algorithms (such as k-NN and SVM) are sensitive to this: a feature with large values will *dominate* the learning process simply because its numbers are larger, not because it is more important.

Feature scaling brings all numerical features to a comparable range.

6.1 Min-Max Normalization

Min-Max scaling (also called *normalization*) transforms each feature to a fixed range, usually $[0, 1]$:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (1)$$

- **Best for:** When you know the data has hard upper and lower bounds, or when the algorithm requires input in $[0, 1]$ (e.g., neural networks with sigmoid output).
- **Caution:** Very sensitive to outliers — a single extreme value compresses all other values into a tiny range.

Code 22: Applying Min-Max scaling to Age and Fare.

```

1 minmax = MinMaxScaler()
2
3 # Fit on training data and transform
4 df[['Age_minmax', 'Fare_minmax']] = minmax.
    fit_transform(df[['Age', 'Fare']])
5
6 print(df[['Age', 'Age_minmax', 'Fare', 'Fare_minmax'
    ]].head())

```

Expected Output

	Age	Age_minmax	Fare	Fare_minmax
0	22.0	0.271	7.250	0.014
1	38.0	0.472	71.283	0.139
2	26.0	0.321	7.925	0.015
3	35.0	0.434	53.100	0.104
4	35.0	0.434	8.050	0.016

6.2 Standardization (Z-score Scaling)

Standardization transforms each feature to have a **mean of 0** and a **standard deviation of 1**:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma} \quad (2)$$

where μ is the column mean and σ is the column standard deviation.

- **Best for:** Most machine learning algorithms, especially those that assume normally distributed input (e.g., linear regression, logistic regression, SVM, PCA).
- **Advantage:** Less sensitive to outliers than Min-Max scaling.

Code 23: Applying Standardization (Z-score scaling) to Age and Fare.

```

1 std_scaler = StandardScaler()
2
3 df[['Age_std', 'Fare_std']] = std_scaler.fit_transform
    (df[['Age', 'Fare']])

```

```

4
5 print(df[['Age', 'Age_std', 'Fare', 'Fare_std']].head
      ())

```

Expected Output

	Age	Age_std	Fare	Fare_std
0	22.0	-0.565	7.250	-0.502
1	38.0	0.663	71.283	0.786
2	26.0	-0.258	7.925	-0.489
3	35.0	0.433	53.100	0.421
4	35.0	0.433	8.050	-0.487

6.3 Comparing the Two Methods Visually

Code 24: Side-by-side comparison of original vs. scaled Age distributions.

```

1 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
2
3 axes[0].hist(df['Age'], bins=30, color='steelblue',
4             edgecolor='white')
5 axes[0].set_title("Original Age")
6 axes[0].set_xlabel("Age")
7
8 axes[1].hist(df['Age_minmax'], bins=30, color='
9             darkorange', edgecolor='white')
10 axes[1].set_title("Age after Min-Max Scaling")
11 axes[1].set_xlabel("Scaled Age [0, 1]")
12
13 axes[2].hist(df['Age_std'], bins=30, color='seagreen',
14             edgecolor='white')
15 axes[2].set_title("Age after Standardization")
16 axes[2].set_xlabel("Z-score")
17
18 plt.suptitle("Effect of Scaling on Age Distribution",
19             fontsize=13, y=1.02)
20 plt.tight_layout()
21 plt.show()

```

Note

Notice that scaling changes the *range* of values but does **not** change the *shape* of the distribution. A right-skewed distribution stays right-skewed after scaling.

For the rest of this lab, we will keep the original **Age** and **Fare** columns for EDA visualizations, as the raw values are more interpretable.

7 Exploratory Data Analysis

With a clean, fully numeric dataset, we can now explore it in depth. EDA uses **statistics** and **visualizations** to answer questions like: What does each feature look like? Are there outliers? Which features are correlated with the target?

7.1 Distribution of Individual Features

Understanding the distribution of each feature reveals skewness, outliers, and natural groupings.

7.1.1 Histograms

A **histogram** groups values into bins and shows how many samples fall into each bin. It reveals the shape of the distribution.

Code 25: Plotting histograms for all numerical features.

```
1 numerical_cols = ['Age', 'Fare', 'SibSp', 'Parch']
2
3 fig, axes = plt.subplots(2, 2, figsize=(12, 8))
4 axes = axes.flatten()
5
6 for i, col in enumerate(numerical_cols):
7     axes[i].hist(df[col], bins=30, color='steelblue',
8                 edgecolor='white')
9     axes[i].set_title(f"Distribution of {col}")
10    axes[i].set_xlabel(col)
11    axes[i].set_ylabel("Count")
```

```

12 plt.suptitle("Histograms of Numerical Features",
    fontsize=14, y=1.01)
13 plt.tight_layout()
14 plt.show()

```

7.1.2 KDE Plots (Kernel Density Estimation)

A **KDE plot** is a smoothed version of the histogram. It estimates the underlying probability density of the data and is useful for comparing distributions across groups.

Code 26: KDE plot of Age split by survival status.

```

1 plt.figure(figsize=(8, 4))
2
3 sns.kdeplot(data=df, x='Age', hue='Survived', fill=
    True, palette={0: 'salmon', 1: 'steelblue'})
4
5 plt.title("Age Distribution: Survived vs. Not Survived
    ")
6 plt.xlabel("Age")
7 plt.ylabel("Density")
8 plt.legend(title="Survived", labels=["No (0)", "Yes
    (1)"])
9 plt.show()

```

Note

What do you observe? Does the age distribution differ between survivors and non-survivors? Think about what patterns you see: younger children seem to have a higher survival density. Can you guess why?

7.2 Categorical Feature Distributions

7.2.1 Bar Charts

Bar charts count the occurrences of each category and are the natural counterpart of histograms for categorical data.

Code 27: Survival count by passenger class.

```

1 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
2
3 # Overall class distribution
4 df['Pclass'].value_counts().sort_index().plot(
5     kind='bar', ax=axes[0], color=['steelblue', '
6         darkorange', 'seagreen'],
7     edgecolor='white')
8 axes[0].set_title("Passenger Count by Class")
9 axes[0].set_xlabel("Passenger Class")
10 axes[0].set_ylabel("Count")
11 axes[0].tick_params(axis='x', rotation=0)
12
13 # Survival rate by class
14 survival_by_class = df.groupby('Pclass')['Survived'].
15     mean()
16 survival_by_class.plot(kind='bar', ax=axes[1], color=[
17     'steelblue', 'darkorange', 'seagreen'],
18     edgecolor='white')
19 axes[1].set_title("Survival Rate by Passenger Class")
20 axes[1].set_xlabel("Passenger Class")
21 axes[1].set_ylabel("Survival Rate")
22 axes[1].tick_params(axis='x', rotation=0)
23 axes[1].set_ylim(0, 1)
24
25 plt.tight_layout()
26 plt.show()

```

Code 28: Survival count by sex using seaborn countplot.

```

1 plt.figure(figsize=(6, 4))
2 sns.countplot(data=df, x='Sex_encoded', hue='Survived'
3     ,
4     palette={0: 'salmon', 1: 'steelblue'})
5 plt.title("Survival Count by Sex")
6 plt.xlabel("Sex (0 = Female, 1 = Male)")
7 plt.ylabel("Count")
8 plt.legend(title="Survived", labels=["No", "Yes"])
9 plt.show()

```

7.3 Box Plots and Outlier Detection

A **box plot** visualises the five-number summary of a distribution: minimum, Q_1 (25th percentile), median (Q_2), Q_3 (75th percentile), and maximum. Points beyond the whiskers are flagged as **outliers**.

The whiskers extend to $1.5 \times \text{IQR}$ from the box, where $\text{IQR} = Q_3 - Q_1$.

Code 29: Box plots of Age and Fare grouped by survival.

```
1 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
2
3 sns.boxplot(data=df, x='Survived', y='Age', palette
4             ={'0': 'salmon', 1: 'steelblue'},
5             ax=axes[0])
6 axes[0].set_title("Age vs. Survival")
7 axes[0].set_xlabel("Survived (0 = No, 1 = Yes)")
8
9 sns.boxplot(data=df, x='Survived', y='Fare', palette
10            ={'0': 'salmon', 1: 'steelblue'},
11            ax=axes[1])
12 axes[1].set_title("Fare vs. Survival")
13 axes[1].set_xlabel("Survived (0 = No, 1 = Yes)")
14
15 plt.suptitle("Box Plots by Survival Status", fontsize
16             =13, y=1.01)
17 plt.tight_layout()
18 plt.show()
```

Code 30: Detecting outliers in Fare using the IQR method.

```
1 Q1 = df['Fare'].quantile(0.25)
2 Q3 = df['Fare'].quantile(0.75)
3 IQR = Q3 - Q1
4
5 lower_bound = Q1 - 1.5 * IQR
6 upper_bound = Q3 + 1.5 * IQR
7
8 outliers = df[(df['Fare'] < lower_bound) | (df['Fare']
9             > upper_bound)]
10 print(f"Q1={Q1:.2f}, Q3={Q3:.2f}, IQR={IQR:.2f}")
11 print(f"Lower bound: {lower_bound:.2f}, Upper bound: {
12       upper_bound:.2f}")
```

```
11 print(f"Number of outliers in Fare: {len(outliers)}")
```

Expected Output

```
Q1=7.91, Q3=31.00, IQR=23.09
Lower bound: -26.72, Upper bound: 65.63
Number of outliers in Fare: 116
```

7.4 Correlation Analysis

Correlation measures how strongly two numerical variables move together. The **Pearson correlation coefficient** r ranges from -1 to $+1$:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (3)$$

- $r \approx +1$: strong positive correlation (both increase together).
- $r \approx -1$: strong negative correlation (one increases as the other decreases).
- $r \approx 0$: little or no linear relationship.

7.4.1 Correlation Heatmap

Code 31: Computing and plotting the correlation matrix.

```
1 # Select only numeric columns
2 numeric_df = df.select_dtypes(include=[np.number])
3
4 corr_matrix = numeric_df.corr()
5
6 plt.figure(figsize=(10, 7))
7 sns.heatmap(corr_matrix,
8             annot=True,           # Show correlation
9             values                 values
10             fmt=".2f",           # Round to 2 decimal
11                                 places
12             cmap='coolwarm',    # Diverging colormap:
13                                 blue=negative, red=positive
```

```

11         center=0,
12         linewidths=0.5,
13         square=True)
14 plt.title("Pearson Correlation Heatmap", fontsize=14)
15 plt.tight_layout()
16 plt.show()

```

Note

Look at the row/column corresponding to **Survived**. Which features have the strongest (positive or negative) correlation with survival? Features with higher absolute correlation are generally more informative for prediction.

7.4.2 Scatter Plots

Scatter plots reveal the *shape* of the relationship between two continuous features, going beyond what a single number can tell you.

Code 32: Scatter plot of Age vs. Fare coloured by survival.

```

1 plt.figure(figsize=(8, 5))
2 colors = df['Survived'].map({0: 'salmon', 1: '
   steelblue'})
3
4 plt.scatter(df['Age'], df['Fare'], c=colors, alpha
   =0.5, edgecolors='none', s=30)
5
6 # Manual legend
7 import matplotlib.patches as mpatches
8 legend_elements = [mpatches.Patch(color='salmon',
   label='Not Survived'),
9                     mpatches.Patch(color='steelblue',
   label='Survived')]
10 plt.legend(handles=legend_elements)
11
12 plt.title("Age vs. Fare (coloured by Survival)")
13 plt.xlabel("Age")
14 plt.ylabel("Fare")
15 plt.show()

```

7.5 Pair Plot

A **pair plot** generates scatter plots for every pair of numerical features and histograms along the diagonal. It is a quick way to scan for relationships across multiple features at once.

Code 33: Pair plot of selected features.

```
1 cols_to_plot = ['Age', 'Fare', 'Pclass', 'SibSp', 'Survived']
2
3 sns.pairplot(df[cols_to_plot], hue='Survived',
4             palette={0: 'salmon', 1: 'steelblue'},
5             plot_kws={'alpha': 0.4, 's': 20},
6             diag_kind='kde')
7 plt.suptitle("Pair Plot of Selected Features", y=1.02,
8             fontsize=13)
9 plt.show()
```

Remember

Pair plots can be slow for datasets with many columns. Restrict to the most relevant features (5–8) to keep runtime manageable.

7.6 Violin Plots

A **violin plot** combines a box plot with a KDE plot, showing both the summary statistics and the full distribution shape.

Code 34: Violin plot of Age by Passenger Class and Survival.

```
1 plt.figure(figsize=(10, 5))
2 sns.violinplot(data=df, x='Pclass', y='Age', hue='Survived',
3             split=True, palette={0: 'salmon', 1: 'steelblue'}, inner='quart')
4 plt.title("Age Distribution by Passenger Class and Survival")
5 plt.xlabel("Passenger Class")
6 plt.ylabel("Age")
7 plt.legend(title="Survived", labels=["No", "Yes"])
8 plt.show()
```

8 Additional Resources

- Pandas Documentation: <https://pandas.pydata.org/docs/>
- Seaborn Gallery (plot examples): <https://seaborn.pydata.org/examples/>
- Matplotlib Documentation: <https://matplotlib.org/stable/contents.html>
- Scikit-learn Preprocessing: <https://scikit-learn.org/stable/modules/preprocessing.html>
- Kaggle Titanic Competition (for further exploration): <https://www.kaggle.com/c/titanic>

9 Tasks

Complete all tasks in your Colab notebook. Each task should be in a separate labelled cell with a Markdown header (e.g., `## Task 1`).

Task 1: Data Loading and Inspection

- Load the Titanic CSV and display the first 10 rows.
- Print the shape, column names, and data types.
- Which columns are numerical vs categorical?
- Print the names of the people who paid more than the average amount in fares.
- Print the percentage of missing values for every column, sorted from highest to lowest.

Task 2: Handling Missing Values

- Drop the `Cabin` column.
- Fill missing `Age` values with the **median**. Compare the distribution of `Age` before and after filling using a histogram. Comment on any difference you observe.
- Fill missing `Embarked` values with the mode.
- Confirm that no missing values remain.

Task 3: Encoding and Scaling

- Apply Label Encoding to the `Sex` column.
- Apply One-Hot Encoding to the `Embarked` column with `drop_first=True`.
- Apply **Min-Max Scaling** to both `Age` and `Fare`. Print the minimum and maximum values of the scaled columns and verify they fall within $[0, 1]$.

Task 4: Exploratory Data Analysis

Using your cleaned and encoded dataset, produce the following visualizations. Each plot must have a **title**, **axis labels**, and a **legend** where applicable.

- (a) **Histogram:** Plot the distribution of **Fare**. Comment on its shape.
- (b) **Bar chart:** Plot the total number of passengers who survived vs. did not survive. Color the bars differently.
- (c) **Box plot:** Create side-by-side box plots of **Fare** for each passenger class (**Pclass**). Which class has the highest median fare? Which class has the most outliers?
- (d) **Correlation heatmap:** Compute and plot the full Pearson correlation matrix. List the top 3 features most correlated with **Survived** (by absolute value).
- (e) **KDE plot:** Plot the KDE of **Age** separately for male and female passengers on the same axes (use the **Sex_encoded** or original **Sex** column before encoding). Do the two groups have similar age distributions?

Task 5: Open-Ended Investigation

Using any combination of the techniques from this lab, answer the following question:

“Was the famous ‘women and children first’ policy reflected in the Titanic survival data?”

Support your answer with at least **two visualizations** and a brief written explanation of what the plots show. There is no single correct answer, your reasoning and evidence matters the most.

10 Submission Instructions

- Complete all tasks in a single Google Colab notebook.
- Name your notebook: **ML_Lab2_YOUR_STUDENT_ID**

- Ensure all cells are executed and all outputs (including plots) are visible.
- Download the notebook as a **Python File** and submit via the course portal.

Remember

Make sure your notebook runs from top to bottom without errors before submitting. Use `Runtime` → `Restart and run all` to verify.